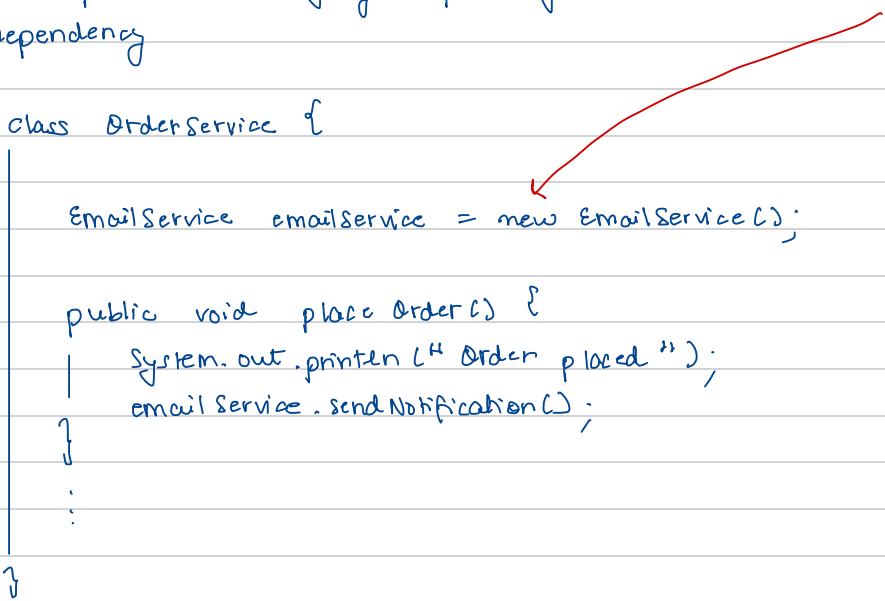
Dependency Injection, IoC & Loose Coupling

→ The problem with tightly coupled systems is that it creates it's own dependency



```
class OrderService {

    EmailService emailService = new EmailService();

    public void placeOrder() {
        System.out.println("Order placed");
        emailService.sendNotification();
    }
    :
}
```

this leads to tight coupling b/w OrderService & EmailService

→ concrete class : A class which has all method definitions with no abstract methods present

→ In Java, we should code for interfaces rather than concrete classes to create loosely coupled designs

```java
package org.kdkbuilds;

import org.kdkbuilds.notification.EmailNotificationService;
import org.kdkbuilds.notification.NotificationService;
import org.kdkbuilds.notification.PopUpNotificationService;
import org.kdkbuilds.notification.SmsNotificationService;

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter choice: ");
        String userInput = scanner.nextLine().strip();
        int choice = Integer.parseInt(userInput);

        NotificationService notification = switch (choice) {
            case 1 -> new EmailNotificationService();
            case 2 -> new PopUpNotificationService();
            default -> new SmsNotificationService();
        };

        OrderService orderService = new OrderService(notification);
        orderService.placeOrder();
    }
}
```

→ In this way, even though OrderService is dependent on NotificationService, we have loosely coupled both of them by injecting the appropriate dependency rather than letting it create the dependency itself.

Advantage

→ Now, OrderService class can focus purely on it's business logic rather than handling it's own dependencies, due to loose coupling

→ This makes unit testing of classes easier

→ We can also say that it has lead to Inversion of Control

Inversion of Control vs Dependency Injection

IOC is the idea / principle to have loosely coupled systems
DI is the actual implementation of that principle

→ In the code-snippet, Main class is orchestrating the creation, management & wiring of dependencies.

→ We could proceed this way, but in a large code-base, the Main file itself would become very complex

→ That is where the Spring Framework steps in and takes that control / responsibility from the developer.

Spring Framework → IOC Container
1. Creates Objects
2. Manages Objects
3. Connects Objects together